

Programación Concurrente

Primer Semestre 2026

Trabajo Práctico

Codificación y decodificación concurrente
mediante Run-Length Encoding (RLE)

Informe

Índice

Índice	2
1. Autoría	3
2. Introducción	3
2.1. Dominio del problema.....	3
2.2. Arquitectura general	3
2.3. Descripción de las clases	4
2.4. Decisiones de diseño.....	5
3. Evaluación	6
3.1. Metodología.....	6
3.2. Entorno de ejecución	6
3.3. Resultados — Tablas de tiempos	6
3.4. Resultados — Gráficos.....	8
4. Análisis	11
4.1. Combinaciones de tiempo mínimo y máximo.....	11
4.2. Discusión.....	11
4.3. Conclusiones.....	12

1. Autoría

Trabajo realizado por:

Nombre y Apellido	E-mail	Legajo / DNI
Lucas Lucarni	lucaslucarni@yahoo.com.ar	44.962.859
Facundo Carnevale	carnevalefacundo@gmail.com	42.833.101
Nicolas Torres	nicotorz.00@gmail.com	42.369.496

2. Introducción

2.1. Dominio del problema

Run-Length Encoding (RLE) es una técnica de codificación que representa secuencias de caracteres repetidos como pares (símbolo, cantidad). En este trabajo, por cada carácter Unicode del archivo de texto original se almacena su codepoint, un entero, junto con la cantidad de veces consecutivas que aparece. Por ejemplo, la cadena "AAAbbbbAA" se codifica como (65,2) (98,5) (65,2).

El archivo codificado se guarda en formato binario, donde cada entero ocupa 32 bits (4 bytes). De este modo, cada par (codepoint, cantidad) ocupa exactamente 8 bytes. La decodificación realiza el proceso inverso, lee el binario de a 8 bytes, interpreta el primer entero como codepoint y el segundo como la cantidad de repeticiones, y reconstruye el texto original.

El trabajo no busca compresión, de hecho, con texto de baja repetición el archivo resultante puede ser mayor que el original, porque cada entero ocupa 4 bytes, sino una codificación correcta que funcione con cualquier carácter Unicode, incluidos emojis y saltos de línea. Para soportar correctamente caracteres fuera del Plano Multilingüe Básico (como los emojis), se trabaja a nivel de codepoints y no de char de Java (que es de 16 bits y no alcanza para esos símbolos).

El objetivo central es realizar la codificación y la decodificación de forma concurrente, el archivo de entrada se reparte entre varios threads que procesan distintas regiones en paralelo. Esto plantea dos desafíos que el programa debe resolver explícitamente. Primero la salida debe preservar el orden del archivo original, y segundo en la codificación, una secuencia de caracteres repetidos que queda partida entre dos threads debe reconocerse como un único bloque (por ejemplo, "bAAAAb" repartido entre dos threads debe producir (98,1) (65,4) (98,1), y no (98,1) (65,2) (65,2) (98,1)).

2.2. Arquitectura general

El programa se estructura como un esquema productor–consumidor con un pool de threads. El thread principal (Main) actúa como productor, carga el archivo en memoria, lo particiona en segmentos y encola una tarea por segmento en un buffer compartido y acotado. Un conjunto fijo de threads trabajadores (RLEWorker) actúa como consumidor, cada worker toma tareas del buffer y las ejecuta. El resultado de cada tarea se deposita en una posición fija de un arreglo de resultados, indexada por el orden del segmento. Cuando todos los workers terminan, Main ensambla los resultados en orden y escribe el archivo de salida.

La sincronización se resuelve con dos monitores, el Buffer, que coordina la entrega de tareas entre productor y consumidores, y el WorkerCounter, que evita que Main finalice mientras queden workers activos.

2.3. Descripción de las clases

Main — Punto de entrada. Valida los argumentos de línea de comandos (archivo de entrada, archivo de salida, modo, cantidad de workers y tamaño del buffer), construye la infraestructura concurrente (Buffer, WorkerCounter y ThreadPool), inicia la medición de tiempo y lanza los workers. Según el modo, delega en runEncode o runDecode, que cargan el archivo, lo particionan, encolan las tareas, esperan la finalización de los workers y disparan el ensamblado final. Finalmente informa el tiempo total de procesamiento.

Buffer — Monitor que implementa una cola FIFO concurrente de capacidad acotada sobre un arreglo circular. El método put bloquea al productor mientras el buffer está lleno, take bloquea al consumidor mientras está vacío. Tras cada inserción o extracción se hace notifyAll para despertar a los hilos que pudieran estar esperando. La capacidad es configurable desde Main.

WorkerCounter — Monitor que lleva la cuenta de los workers activos, inicializado con la cantidad total de threads lanzados. El método esperarFin bloquea al hilo llamante (Main) mientras queden workers activos. workerTermino decrementa el contador y, cuando llega a cero, hace notifyAll para despertar a Main. Funciona, en esencia, como una barrera de finalización que garantiza que Main no continúe hasta que el último worker haya terminado.

Task<T> / EncodeTask / DecodeTask — Task<T> es una clase abstracta que implementa Runnable y guarda el orderIndex (la posición del segmento en la secuencia total).

EncodeTask aplica RLE sobre un segmento de codepoints y produce un objeto ResultadoEncode. DecodeTask interpreta un rango de pares del arreglo de bytes (usando ByteBuffer.getInt y StringBuilder.appendCodePoint, lo que permite reconstruir emojis correctamente) y produce el String del segmento. Cada tarea escribe su resultado en la posición orderIndex del arreglo de resultados.

ResultadoEncode — Estructura que almacena el resultado de una EncodeTask separado en tres partes, el primer run del segmento (primerRun), los runs intermedios ya cerrados de forma definitiva (medio) y el último run del segmento (ultimoRun), más el índice del segmento. Esta separación es la que permite fusionar correctamente los bordes entre segmentos adyacentes durante el ensamblado (ver 2.4).

RLEWorker — Thread trabajador. En un bucle, toma tareas del buffer y las ejecuta hasta recibir una PoisonPill. Captura PoisonException para salir del bucle de forma controlada y, en el bloque finally, siempre notifica su finalización al WorkerCounter (incluso ante una interrupción inesperada).

PoisonPill / PoisonException — PoisonPill es una Task especial cuyo run lanza una PoisonException. Sirve como señal de terminación, cuando un worker toma una PoisonPill, la excepción lo hace salir de su bucle. Se encola una PoisonPill por cada worker, de modo que todos terminen.

ThreadPool — Encapsula la creación de los RLEWorker (recibiendo el buffer y el contador compartidos) y los inicia con el método iniciar.

2.4. Decisiones de diseño

Apertura y partición del archivo. El archivo se sube completo a memoria antes de procesar. En codificación se lee con `Files.readString` en UTF-8 y se convierte a un arreglo de codepoints con `codePoints()`; en decodificación se lee con `Files.readAllBytes` y se interpreta como pares de 8 bytes. La partición divide la entrada en tantos segmentos como workers de forma balanceada, se reparte el cociente entre todos y se distribuye el resto entre los primeros segmentos. Se crea como máximo un segmento por unidad disponible, de modo que con entradas más chicas que la cantidad de workers no se generan segmentos vacíos.

Preservación del orden. Aunque los workers procesan los segmentos en paralelo y pueden terminar en cualquier orden, cada tarea conoce su `orderIndex` y escribe su resultado en esa posición fija de un arreglo pre-dimensionado. El ensamblado recorre ese arreglo en orden de índice, por lo que la salida respeta siempre la secuencia del archivo original, independientemente de qué worker terminó primero. El orden no depende del FIFO del buffer (que solo garantiza la equidad en la entrega de tareas), sino de esta indexación explícita.

Fusión de bordes en la codificación. Para evitar que un mismo símbolo quede partido en dos runs por haber caído en el borde de dos segmentos, cada `EncodeTask` no entrega una lista plana de runs, sino que separa explícitamente su primer run, sus runs intermedios y su último run. Los runs intermedios están "cerrados" de forma definitiva (no pueden fusionarse con nada), pero el primer y el último run de cada segmento son candidatos a fusionarse con el segmento vecino. Durante el ensamblado (que es secuencial, en `Main`, una vez que todos los workers terminaron), se va construyendo la lista final antes de agregar el primer run de un segmento, se compara su codepoint con el del último run ya acumulado; si coinciden, se suman las cantidades en lugar de agregar un run nuevo.

Un caso particular es el de un segmento cuyo contenido es un único run (por ejemplo, un tramo de "AAAA" que cae enteramente dentro de un segmento, o una secuencia repetida que empieza antes y termina después del segmento). En ese caso, `primerRun` y `ultimoRun` son el mismo run, y el ensamblado lo detecta para no agregarlo dos veces. Así, una secuencia repetida que atraviesa varios segmentos se fusiona correctamente en un único par. Por ejemplo, "bAAAAb" repartido en "bAA" y "AAb" produce (98,1) (65,4) (98,1), como exige el enunciado.

Terminación mediante PoisonPill. Una vez encoladas todas las tareas reales, `Main` encola una `PoisonPill` por cada worker. Como el buffer es FIFO y las píldoras se encolan después de todas las tareas, ningún worker puede recibir una píldora antes de que se hayan repartido todas las tareas de trabajo; y como hay exactamente una píldora por worker, todos terminan de forma ordenada. La terminación por excepción (`PoisonException`) mantiene el bucle del worker simple y centraliza la lógica de salida.

3. Evaluación

3.1. Metodología

Se evaluó el rendimiento del programa variando el tamaño del texto, el grado de repetición de caracteres, el modo de operación y la cantidad de threads. Las dimensiones de prueba son:

- **Tamaño del texto:** pequeño (< 1 KB), mediano ($\approx$ 512 KB) y grande ($\approx$ 2 MB).
- **Repetición de caracteres:** baja (texto en lenguaje natural repetido) y alta (texto con largas secuencias de caracteres repetidos).
- **Modo:** codificación y decodificación.
- **Cantidad de threads:** 1, 4, 8 y 16.

Adicionalmente, se verificó el funcionamiento con un ejemplo pequeño (menos de 128 bytes) que incluye caracteres repetidos, emojis y saltos de línea, comprobando que al codificar y luego decodificar se recupera exactamente el texto original (roundtrip).

El tamaño del buffer se mantuvo fijo en todas las pruebas, con un valor de 16 slots. Cada combinación se ejecutó varias veces y se reportó el tiempo promedio para reducir el ruido de medición. El tiempo se mide con `System.currentTimeMillis()`, desde el instante previo al inicio de los threads hasta que el último worker termina.

3.2. Entorno de ejecución

Todas las pruebas se ejecutaron en el mismo equipo, con las siguientes características:

Procesador	<i>AMD FX-8320E (8 núcleos físicos y 8 hilos de ejecución)</i>
Memoria RAM	<i>16GB DDR3</i>
Sistema Operativo	<i>Windows 10 Pro</i>
Versión de Java	<i>OpenJDK 21</i>

3.3. Resultados — Tablas de tiempos

(Los tiempos se expresan en segundos)

Tabla 1 — Codificación, texto pequeño (< 1 KB)

Repetición	1 thread	4 threads	8 threads	16 threads
Baja	0.053	0.060	0.059	0.076
Alta	0.051	0.062	0.056	0.063

Tabla 2 — Codificación, texto mediano ($\approx$ 512 KB)

Repetición	1 thread	4 threads	8 threads	16 threads
Baja	0.190	0.213	0.242	0.230
Alta	0.076	0.095	0.114	0.108

Tabla 3 — Codificación, texto grande ($\approx$ 2 MB)

Repetición	1 thread	4 threads	8 threads	16 threads
Baja	0.591	0.527	0.485	0.587
Alta	0.090	0.136	0.182	0.195

Tabla 4 — Decodificación, texto pequeño (< 1 KB)

Repetición	1 thread	4 threads	8 threads	16 threads
Baja	0.050	0.056	0.053	0.069
Alta	0.049	0.064	0.059	0.055

Tabla 5 — Decodificación, texto mediano (≈ 512 KB)

Repetición	1 thread	4 threads	8 threads	16 threads
Baja	0.099	0.126	0.206	0.224
Alta	0.080	0.118	0.155	0.155

Tabla 6 — Decodificación, texto grande (≈ 2 MB)

Repetición	1 thread	4 threads	8 threads	16 threads
Baja	0.150	0.169	0.277	0.314
Alta	0.107	0.146	0.171	0.190

3.4. Resultados — Gráficos

Se presentan seis gráficos, uno por cada combinación de tamaño y modo. En cada uno, el eje X representa la cantidad de threads (1, 4, 8, 16) y el eje Y el tiempo de ejecución en segundos. Cada gráfico contiene dos curvas, una para baja repetición y otra para alta repetición.

Gráfico 1 — Codificación, texto pequeño

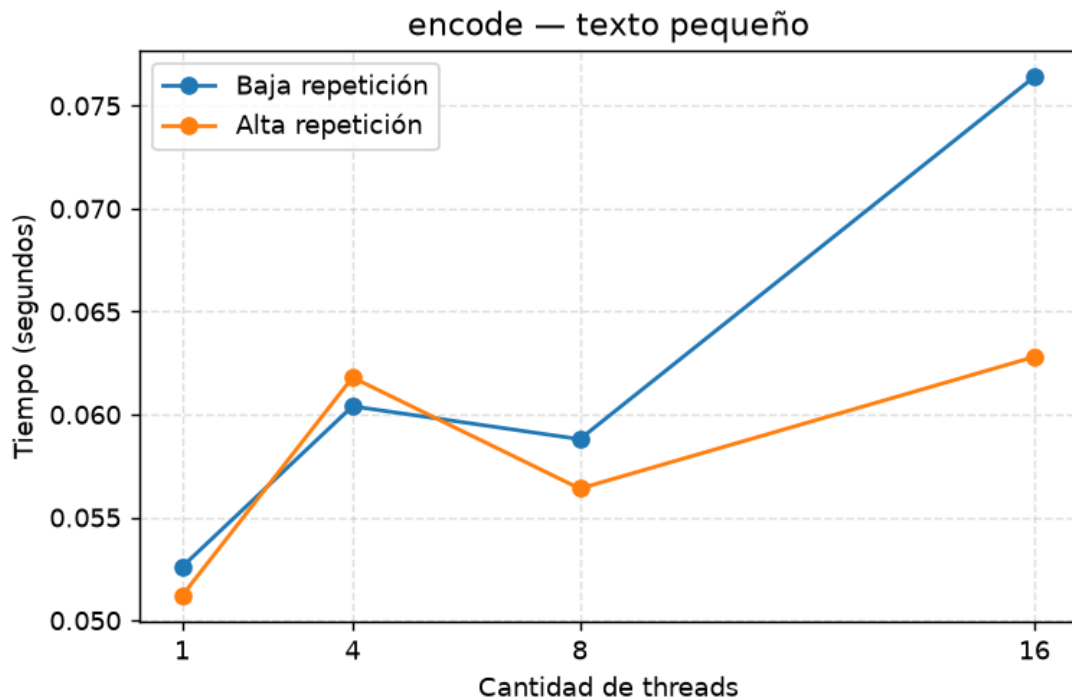


Gráfico 2 — Codificación, texto mediano

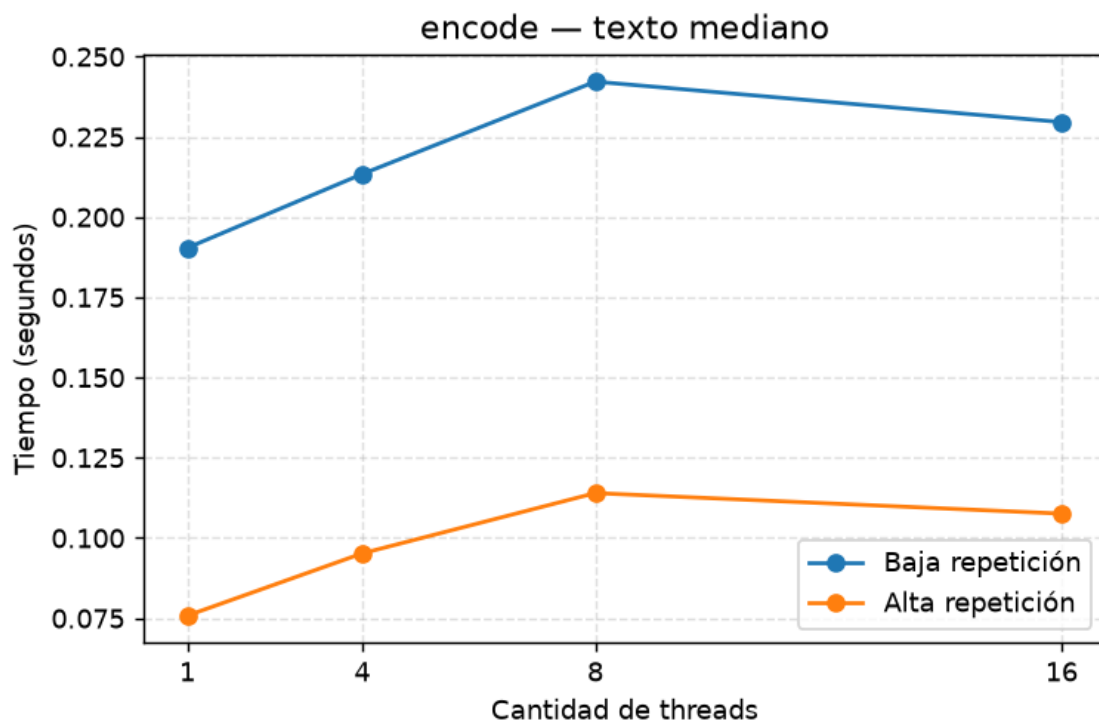


Gráfico 3 — Codificación, texto grande

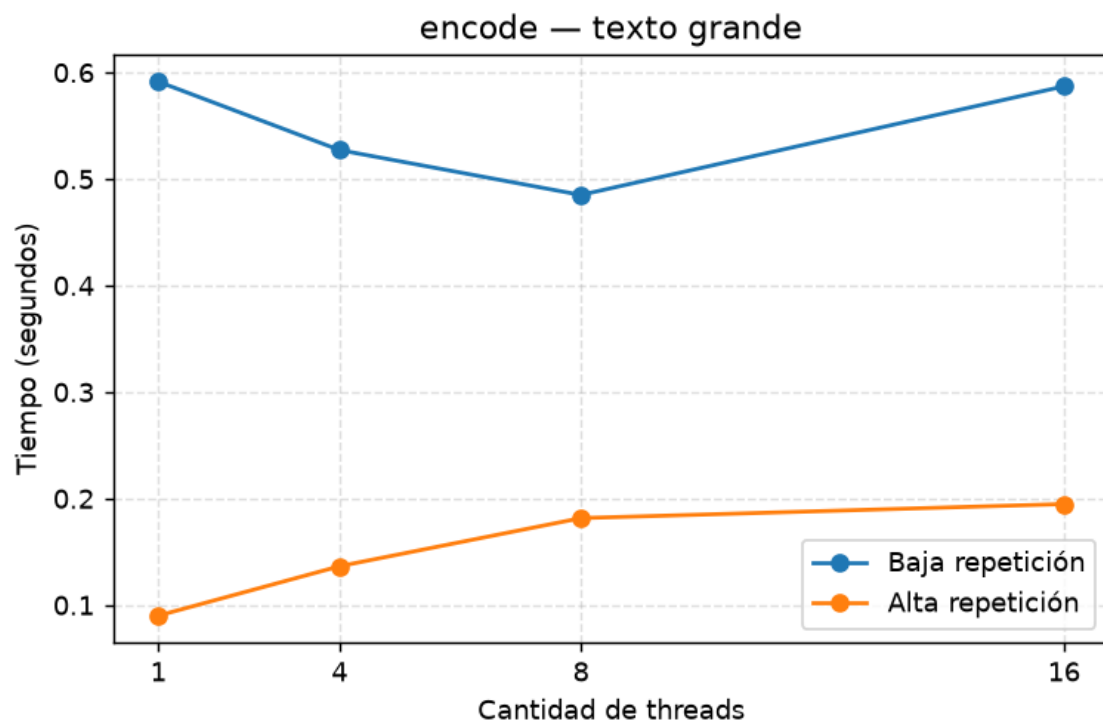


Gráfico 4 — Decodificación, texto pequeño

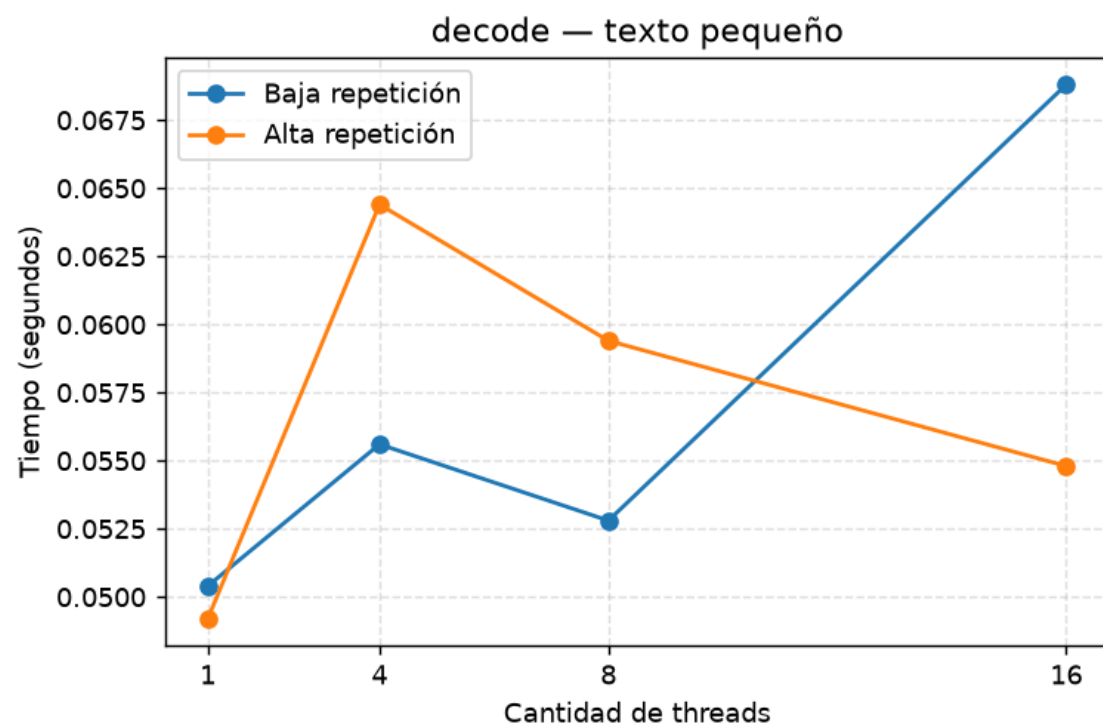


Gráfico 5 — Decodificación, texto mediano

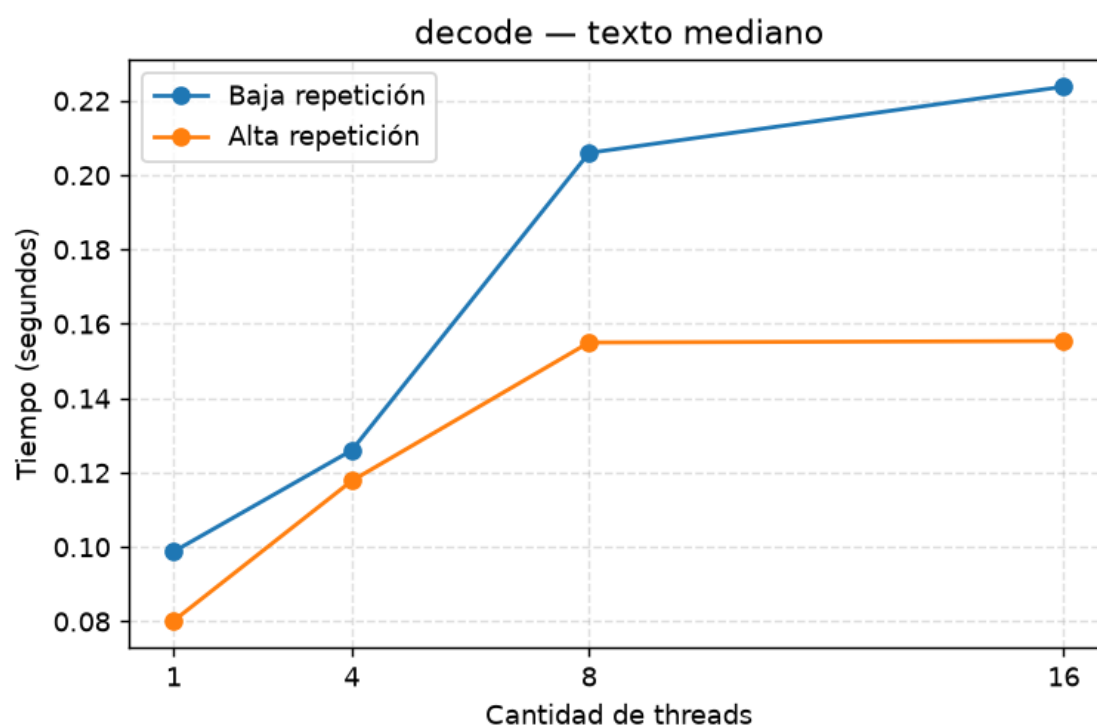
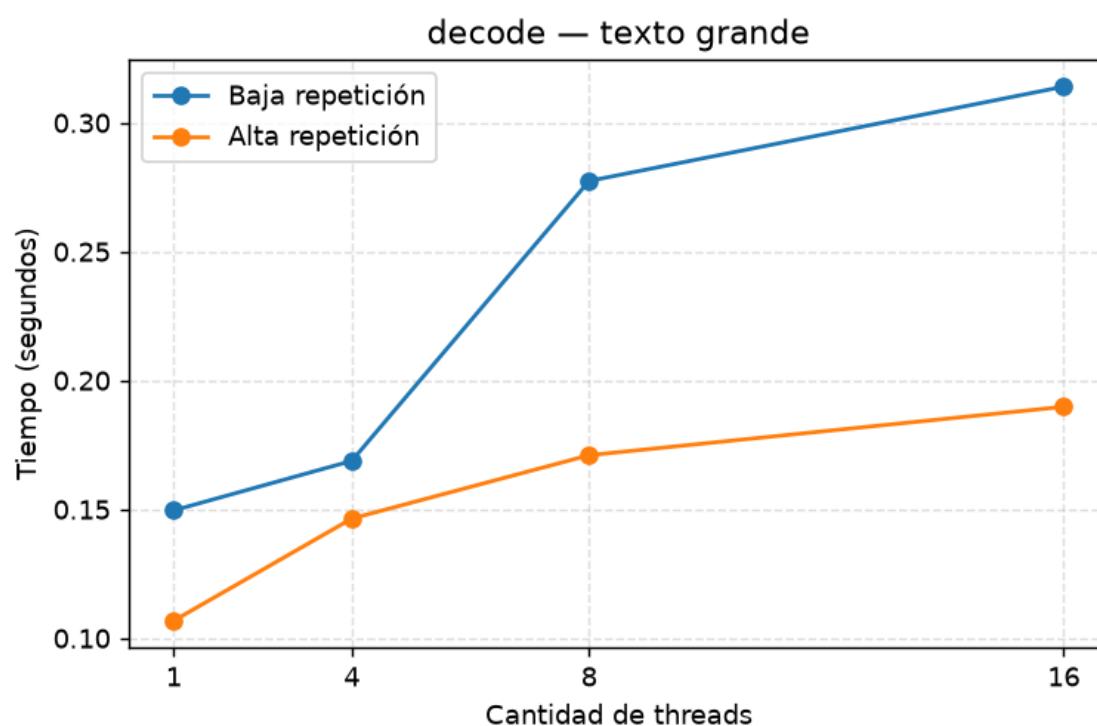


Gráfico 6 — Decodificación, texto grande



4. Análisis

Esta sección discutirá los tiempos obtenidos en la evaluación. Todas las mediciones se realizaron en el mismo equipo, promediando varias ejecuciones por combinación. En los casos más chicos, la diferencia entre el tiempo promedio y el mínimo observado es comparable al propio valor medido, lo que indica una variabilidad considerable. Por eso, las conclusiones se apoyan en las tendencias generales y no en diferencias de pocos milisegundos.

4.1. Combinaciones de tiempo mínimo y máximo

Para cada tamaño de archivo, la combinación de modo y cantidad de threads con la que se obtuvo el menor y el mayor tiempo de ejecución fue:

Tamaño	Tiempo mínimo	Tiempo máximo
Pequeño	decode, alta repetición, 1 thread — 0,049 s	encode, baja repetición, 16 threads — 0,076 s
Mediano	encode, alta repetición, 1 thread — 0,076 s	encode, baja repetición, 8 threads — 0,242 s
Grande	encode, alta repetición, 1 thread — 0,090 s	encode, baja repetición, 1 thread — 0,591 s

Un dato llamativo es el del archivo grande, tanto el mínimo como el máximo se dan con 1 thread, diferenciándose únicamente en el grado de repetición. Esto es el factor que más impacta en el tiempo y no es la cantidad de threads, sino las características del texto y, por lo tanto, la cantidad de runs a procesar.

4.2. Discusión

¿Qué modo resulta más eficiente, codificación o decodificación?

No hay un ganador absoluto, depende del grado de repetición del texto. En textos de baja repetición, la codificación es claramente la operación más costosa, porque RLE genera casi un run por carácter, hay que construir y almacenar una gran cantidad pares y escribir un binario que incluso es más grande que el texto original. En textos de alta repetición, la codificación se vuelve muy barata, mientras que la decodificación sigue teniendo que materializar todo el texto de salida carácter por carácter, por lo que pasa a ser levemente más lenta. En síntesis, la operación más pesada es siempre la que produce más datos, codificar cuando el texto casi no se repite, y la salida de texto sí se repite mucho.

¿Es cierto que aumentar la cantidad de threads siempre mejora el rendimiento?

No. En casi todas las combinaciones, agregar threads dejó el tiempo igual o lo empeoró. El único caso con una mejora real fue la codificación del archivo grande de baja repetición, que bajó de 0,591 s (1 thread) a 0,485 s (8 threads), para luego volver a subir a 0,587 s con 16 threads. En el resto, los tiempos se mantuvieron planos o crecieron de forma sostenida.

¿El comportamiento al aumentar los threads es el mismo cuando cambia el tamaño del texto?

No completamente. En el archivo pequeño (600 bytes) no hay prácticamente nada para paralelizar, así que sumar threads solo lo empeora, el peor tiempo se da casi siempre con 16 threads. En el mediano, los threads tienden a perjudicar. Recién en el grande, y solo para la codificación de baja repetición, aparece un beneficio. La regla general parece ser que cuanto mayor es el volumen de trabajo de CPU realmente paralelizable, más chance

tienen los threads de ayudar, a diferencia que con cargas chicas o dominadas por E/S, lo empeoran.

¿La situación es igual para la codificación que para la decodificación?

No. La codificación fue el único modo que mostró un aumento de velocidad con threads (en el caso grande/baja). La decodificación casi nunca se benefició y, de hecho, anti-escaló de forma marcada, mediano/baja pasó de 0,099 s a 0,224 s y grande/baja de 0,150 s a 0,314 s al ir de 1 a 16 threads. Una razón probable es que el ensamblado de la decodificación concatena en Main las cadenas de cada segmento, con más threads hay más segmentos, más Strings intermedios que asignar y copiar, mientras que el trabajo de expandir cada par es trivial. El costo extra de coordinación y de la concatenación secuencial crece sin un trabajo paralelo que lo justifique.

¿Influye el grado de repetición de los caracteres?

Sí, es el factor de mayor impacto. Cambia tanto la magnitud de los tiempos como qué operación es la pesada. El ejemplo más claro es la codificación del archivo grande, 0,090 s con alta repetición contra 0,591 s con baja repetición. Esto se debe a que la cantidad de runs depende directamente de cuánto se repiten los caracteres, no del tamaño en bytes del archivo.

4.3. Conclusiones

Para los tamaños evaluados, el programa está dominado por sus partes secuenciales, lectura del archivo a memoria, ensamblado en orden y escritura de la salida, y por el costo de sincronización del buffer. En consecuencia, aumentar la cantidad de threads rara vez mejoró el rendimiento y, en varios casos, lo empeoró de forma notable, especialmente en la decodificación.

El único escenario donde la concurrencia rindió fue el de mayor carga de CPU paralelizable, codificar un archivo grande de baja repetición, con un óptimo en torno a 4-8 threads y un retroceso en 16.

El grado de repetición del texto resultó más determinante que la cantidad de threads, y el modo más eficiente dependió de él. Para observar una mejora más consistente con la concurrencia, habría que trabajar con archivos sustancialmente más grandes y/o reducir el costo de coordinación. Finalmente, dado que varios tiempos están por debajo de 0,1 s, conviene interpretarlos con cautela, a esa escala, la planificación del sistema operativo puede introducir un ruido del mismo orden que las diferencias observadas.